game-3d starter (three.js + Rapier)

A complete, small 3D game — **Sky Cores**: run, jump and dash across floating islands to collect
every energy core before the clock runs out, avoiding patrol drones. It exists to be *changed*: the engine layer is fixed scaffolding, the game layer is the part you rewrite.

Bash

```
pnpm install
pnpm dev      # http://localhost:5173
pnpm build    # typecheck + production build + JS/font budgets + total-size rep
pnpm test     # unit tests (rules, save, i18n, level)
pnpm smoke    # after build: headless browser playthrough + screenshots in shot
```

Layout

Path	Role	Change it?
`src/engine/loop.ts`	Fixed 60 Hz simulation + interpolated rendering	Rarely
`src/engine/input.ts`	Keyboard/mouse, gamepad, touch → one action state	Add actions here
`src/engine/physics.ts`	Rapier world, collider helpers, render interpolation	Rarely
`src/engine/renderer.ts`	WebGL renderer, quality presets, bloom	Rarely
`src/engine/audio.ts`	Music/SFX buses, synthesized SFX, `load()` for files	Add sounds
`src/engine/save.ts`	Versioned localStorage save + leaderboard	Add fields + parsing
`src/engine/i18n.ts`	en / zh-CN, browser detection, saved choice wins	Rarely
`src/engine/assets.ts`	Cached GLTF/texture loading with progress	Use it for models
`src/game/config.ts`	Source defaults and active gameplay configuration	Yes

`src/game/tuning.ts`	Tweak catalog, atomic validation and activation boundaries	Yes
`src/game/rules.ts`	Pure score/lives/clock/win rules (unit tested)	Yes
`src/game/world.ts`	Level layout, sky, lights, islands, stones, lifts	Yes
`src/game/player.ts`	Character controller + game feel + procedural model	Yes
`src/game/*.ts`	Camera, cores, drones, particles, scene orchestration	Yes
`src/ui/` , `src/styles/main.css`	HTML/CSS title, HUD, pause, settings, results	Yes
`src/i18n/*.json`	All player-facing text (both files, same keys)	Yes

Rules for changes

- **Simulation in `step()` , visuals in `render()` / `animate()` .** Gameplay state only changes in fixed steps; read presses there with `input.consume(action)` , never `pressed()` .
- **Rules stay pure.** Scoring, win/lose and progression go in `rules.ts` with tests; scene code reports events and reads state.
- **Every visible string is an i18n key** in both `en.json` and `zh-CN.json` (a test enforces matching keys and placeholders). English and Chinese use the bundled Manus CC0 fonts in `public/fonts/` ;
  if you add new Chinese text, check it renders (missing glyphs fall back to system fonts).
- **UI is HTML/CSS**, not canvas. Keep the game look: display font, outlined text, hard offset shadows, skewed buttons, notched panels. Menus must stay keyboard/gamepad navigable
  (`data-nav` on focusable controls).
- **Colliders come from the helpers** in `physics.ts` with the same sizes as the meshes.
- Keep `npm run build` within the JS/font budgets (`scripts/check-size.mjs`) and `npm run smoke` green.
  Total game asset size is reported without a hard limit.

Controls

Keyboard/mouse: WASD move, mouse look (click to capture), Space jump (hold = higher), Shift
sprint, F or left click dash, Esc pause. Gamepad: left stick, right stick, A jump, X/RB dash, LB sprint, Start pause. Touch: left-side stick, right-side drag to look, JUMP / DASH buttons.

Credits

Fonts: ManusCC0 (regular, medium, bold) and ManusCC0 Sans CJK SC — CC0 1.0.
The bundled Chinese font preserves its 6,547-codepoint repertoire; no font download is required.
See `public/fonts/ManusCC0-LICENSE.txt`.
Libraries: three.js (MIT), Rapier (Apache-2.0).

Preview Tweak

`src/game/tuning.ts` is this game's parameter manager. The Addon panel reads its catalog; `config.ts` owns source defaults and the active configuration used by gameplay. When adapting the game, keep only controls with actual gameplay consumers, and preserve
stable IDs for unchanged parameters. Define ranges, units and the correct activation boundary. Register through `scripts/manus-tuning/adapter.js` only under `import.meta.env.DEV`. Keep the Vite plugin and `scripts/__manus__/` helpers intact; production builds exclude the browser adapter and never inject the bridge.

Apply changes preview state only. `LIVE` parameters are read in the next game update; `NEXT_ACTION` parameters activate at the relevant jump/dash start; `NEXT_RUN` parameters activate in `Game.start`. Model/collider size and gravity are not exposed by this version. Do not mutate the source defaults or persist Apply to localStorage. Save with Manus checks the snapshot against the current catalog and edits `DEFAULT_CONFIG`, then uses the normal Three.js Web build/checkpoint workflow. Keep existing gameplay parameter managers when extending an older project; do not create a competing store.

For server-authoritative multiplayer, expose local cosmetic controls only until gameplay parameters have explicit server-side development support. The descriptor's GAMEPLAY flag is a classification, not server authorization or ranked-score enforcement.

The preview plugin reads the ignored `.manus-webdev/preview-identity.json` v1 projection written by the Runtime when initializing, attaching or restoring this project. It contains only the resource ID and canonical project directory; private Host bindings and starter receipts are not template dependencies. Missing identity disables the bridge.
The manager latches `tuning.unranked` when non-default GAMEPLAY values become active

and
resets it only at the next run using that run's active values. `Game` snapshots this onto `RunState.unranked`; local results cannot save such runs. Preserve this guard when changing scoring. When adding online leaderboards, skip submissions for unranked runs and enforce eligibility in the authoritative server too; client flags are not proof of a fair score.